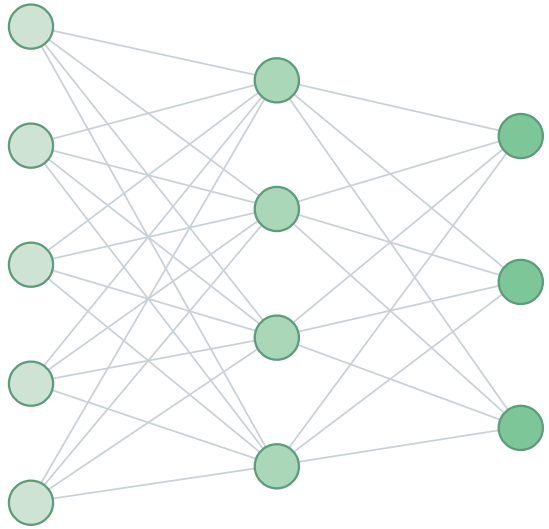
Podstawy sieci neuronowych

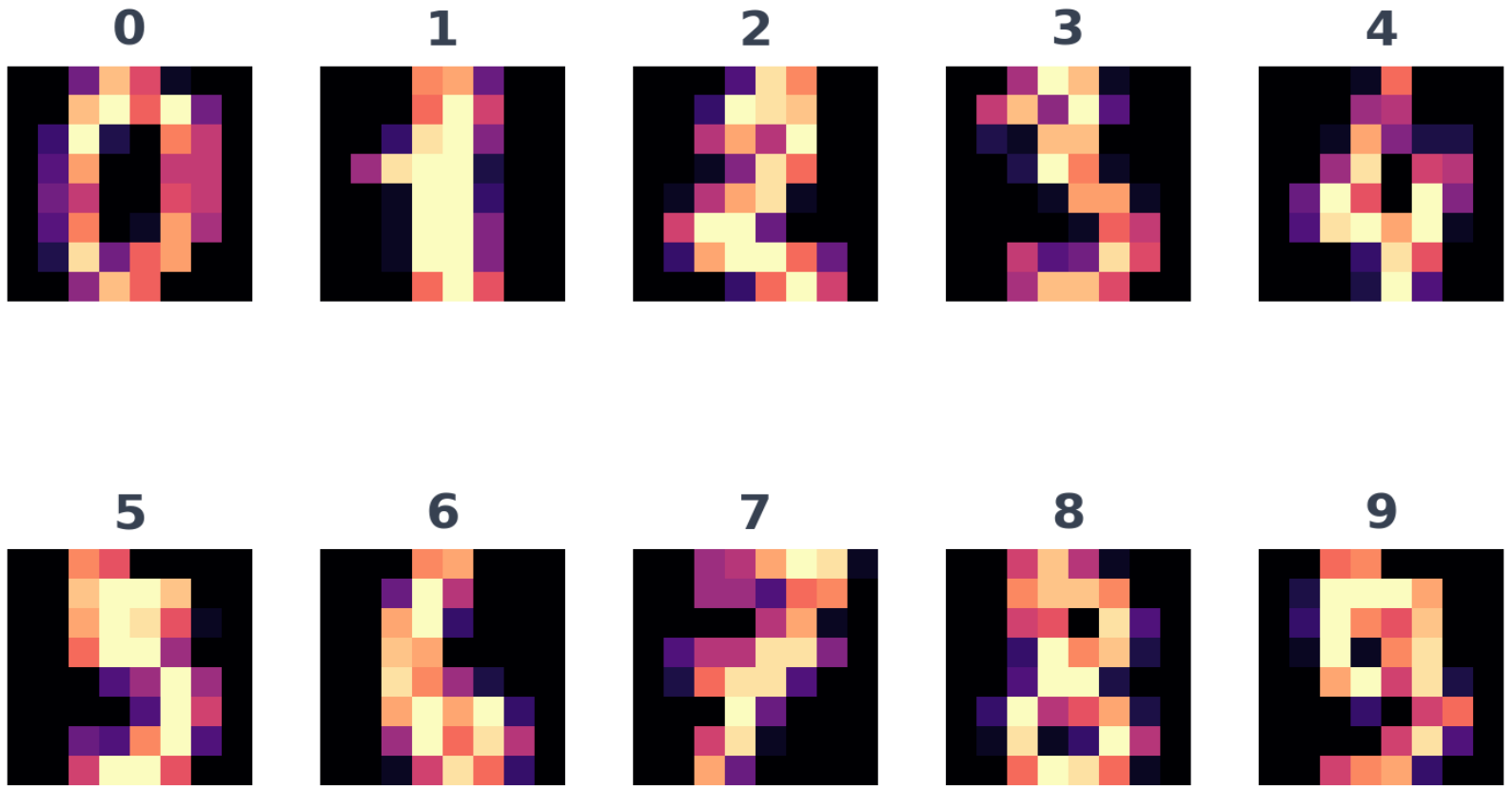


Agnieszka Grala

17.10.2026

Olimpiada Sztucznej Inteligencji - szkolenia

Czy komputer potrafi rozpoznać cyfrę?



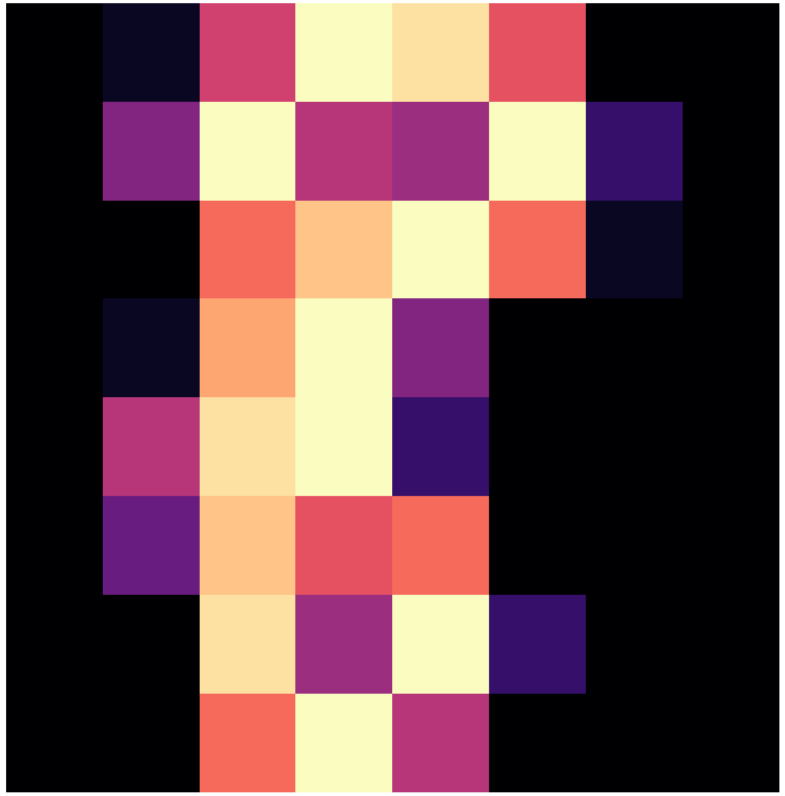
Widzimy:

kształt
i znaczenie

Komputer dostaje jedynie liczby.

Obraz 8 × 8 = 64 liczby

Obraz 8 × 8



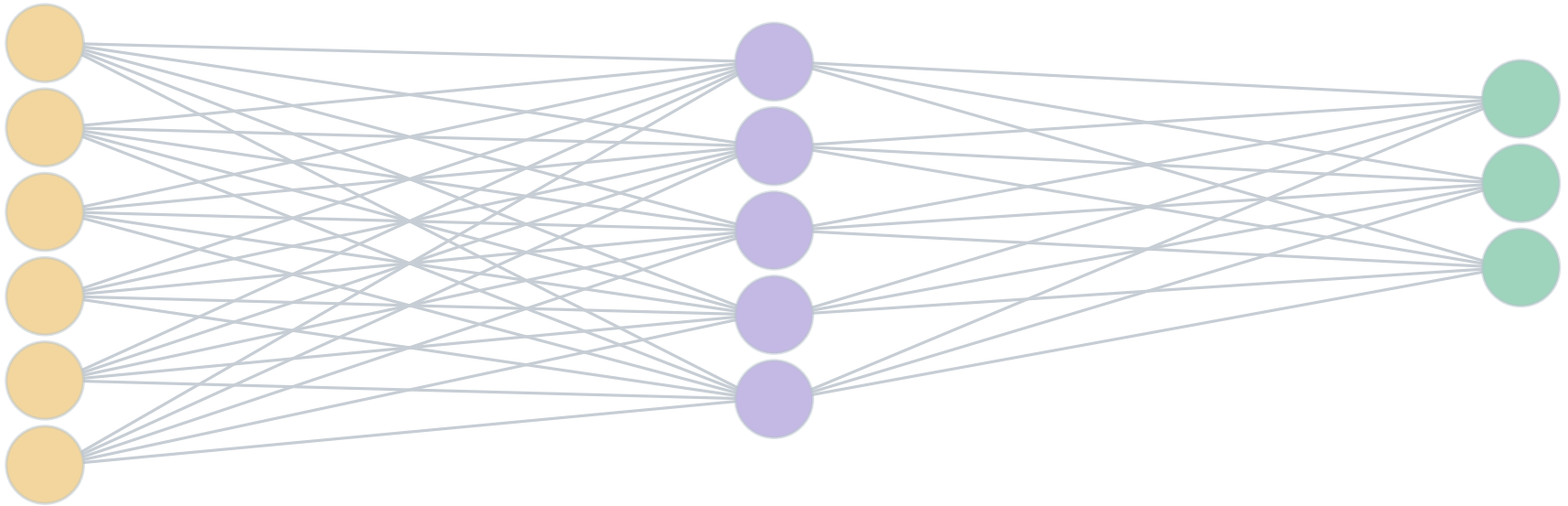
64 liczby od 0 do 16

0	1	9	16	15	10	0	0
0	6	16	8	7	16	3	0
0	0	11	14	16	11	1	0
0	1	13	16	6	0	0	0
0	8	15	16	3	0	0	0
0	5	14	10	11	0	0	0
0	0	15	7	16	3	0	0
0	0	11	16	8	0	0	0

MLP

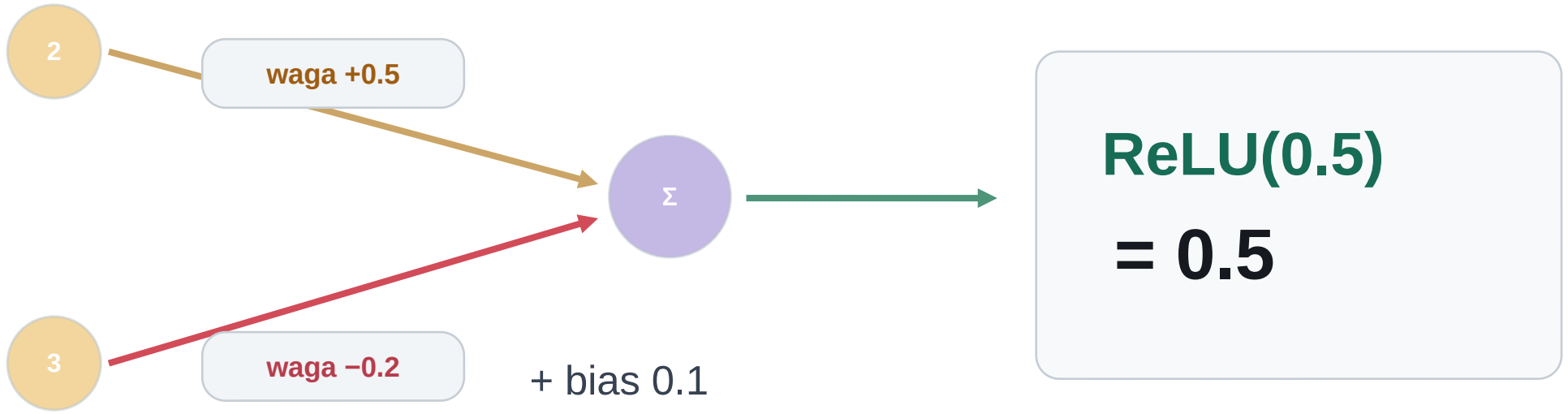
Zamieniamy tabelę pikseli na wektor 64 liczb, np. od 0 do 1.

Sieć jest funkcją, która zamienia wejście na wynik



64 piksele **Warstwa ukryta** **Wyniki 0–9**

Jeden neuron: suma ważona + bias



$$z = w_1x_1 + w_2x_2 + b$$

Waga: które wejście jest ważne? Bias: przesunięcie

Waga dodatnia

Większe wejście zwykle zwiększa sumę w neuronie.

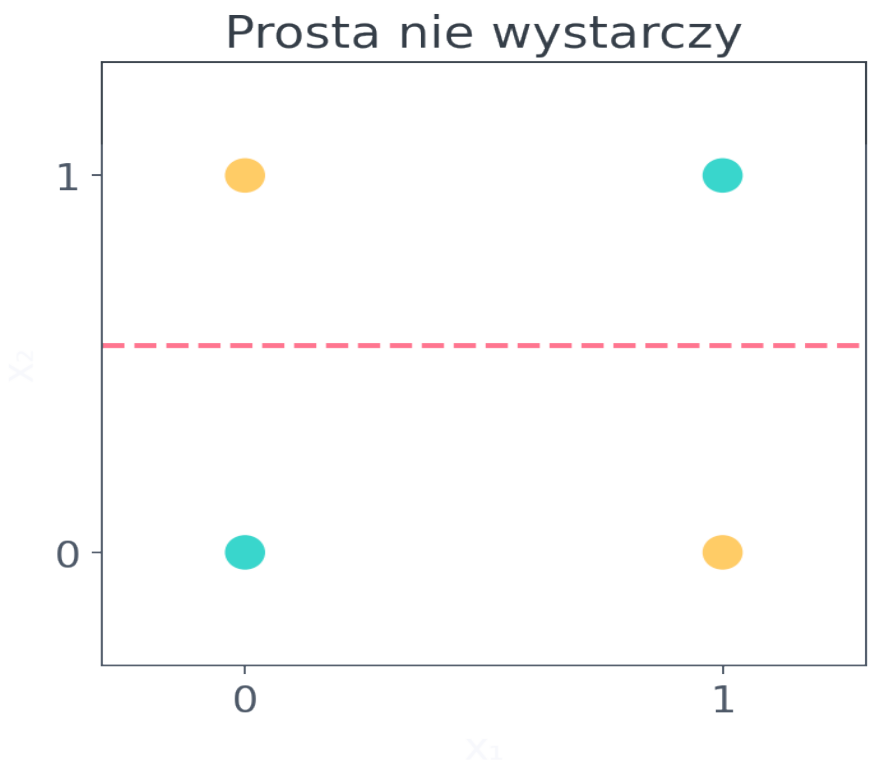
Waga ujemna

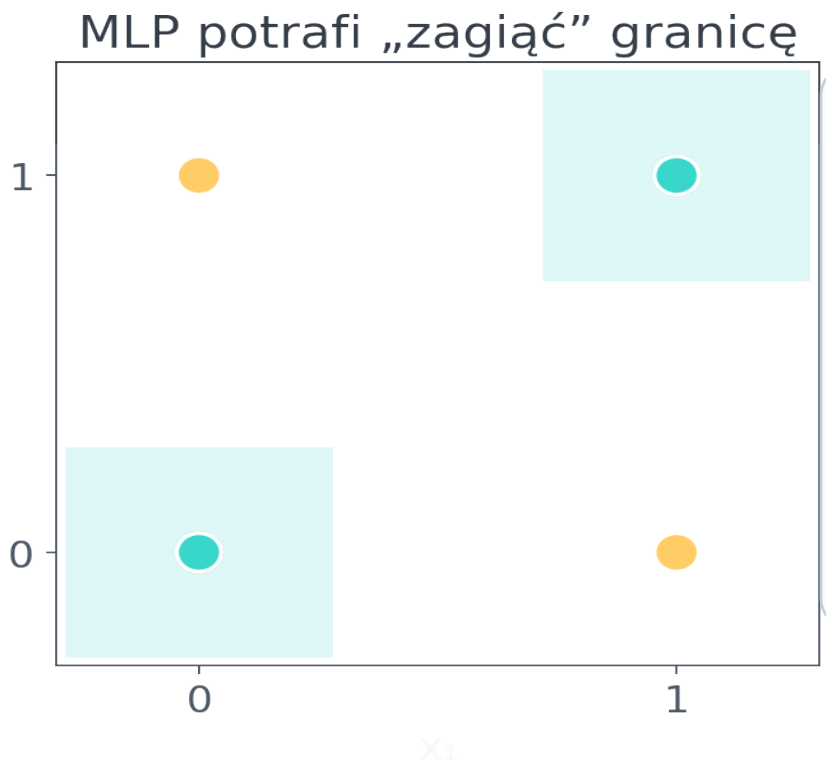
Większe wejście zmniejsza sumę.

Bias

Dodajemy stałą; neuron może reagować wcześniej lub później.

Jeden neuron ma ograniczenia

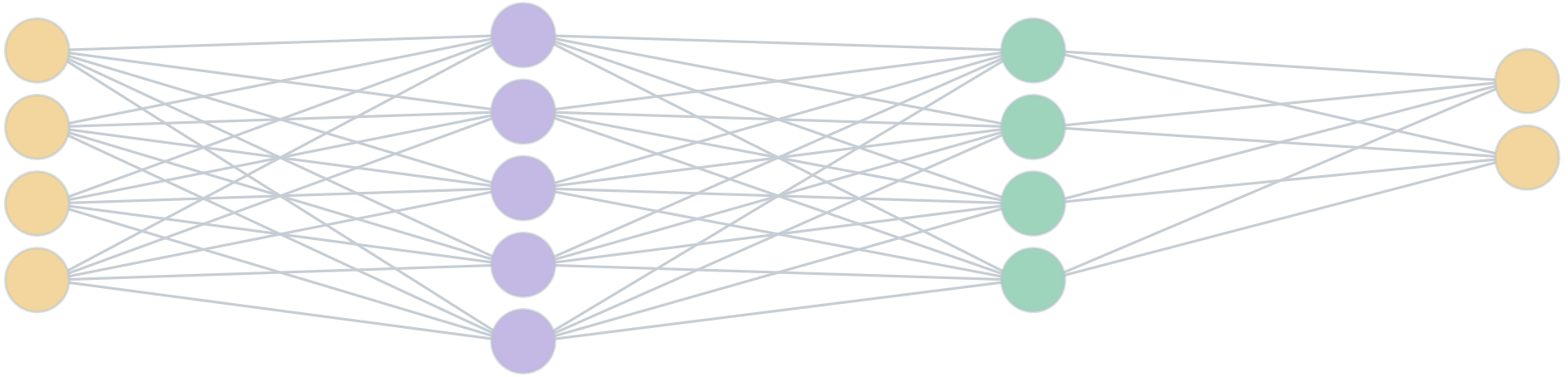




XOR

Tego układu nie rozdzieli jedna prosta.

MLP = wiele neuronów połączonych w warstwy



wejście | warstwa 1 | warstwa 2 | wyjście

Forward pass: wynik idzie od lewej do prawej

Krok 1

Obliczamy sumy ważone:
$z = Wx + b$

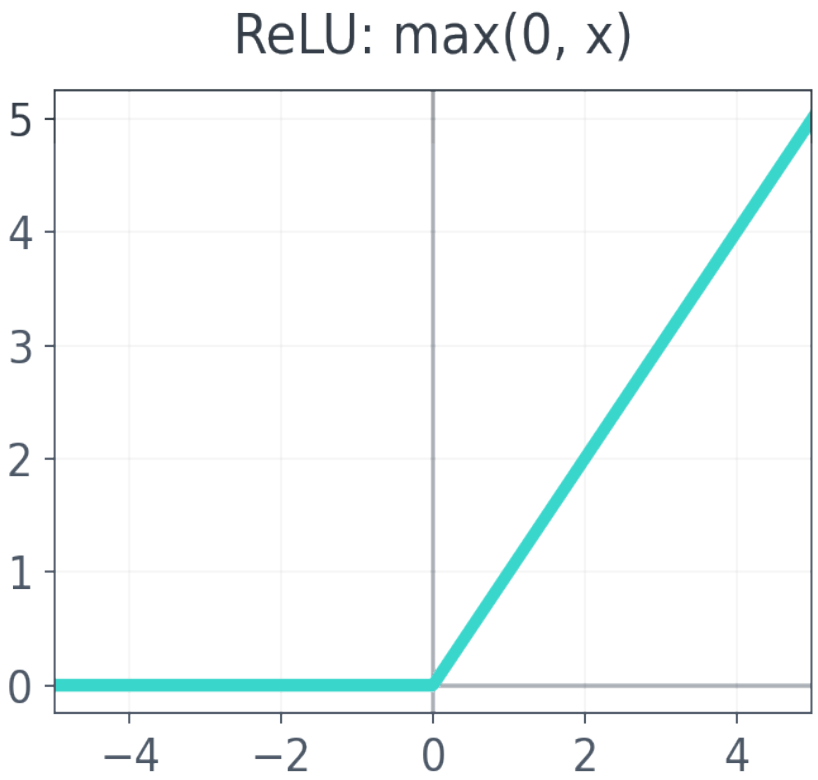
Krok 2

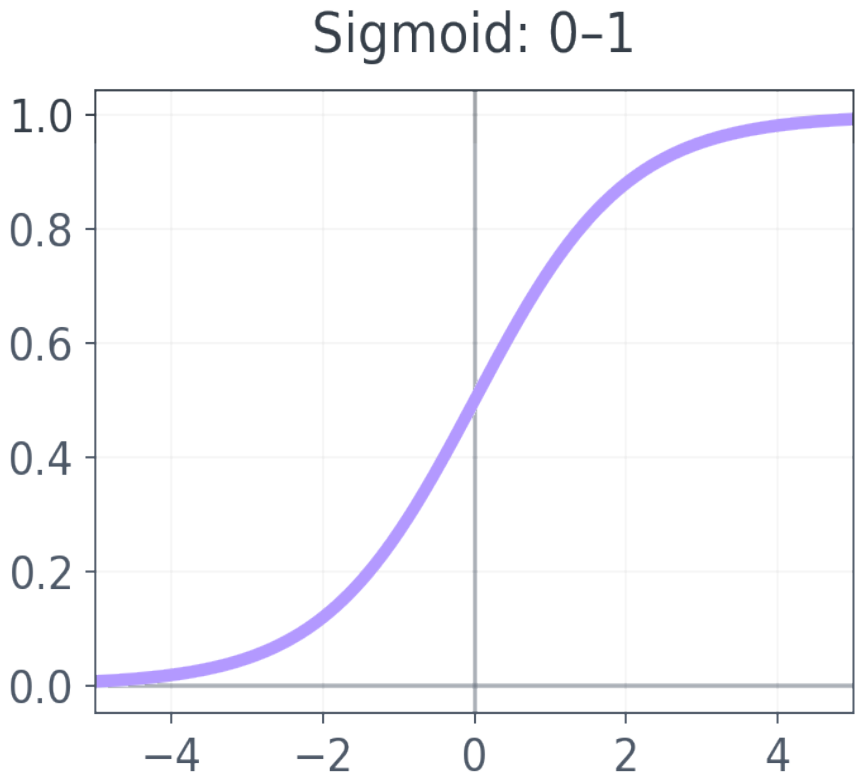
Przepuszczamy je przez ReLU:
$a = \max(0, z)$

Krok 3

Następna warstwa daje wyniki 10 cyfr.

Bez nieliniowości trudno budować złożone modele





Po co?

ReLU tworzy nieliniowość. Pozwala łączyć proste elementy w trudniejsze wzorce.

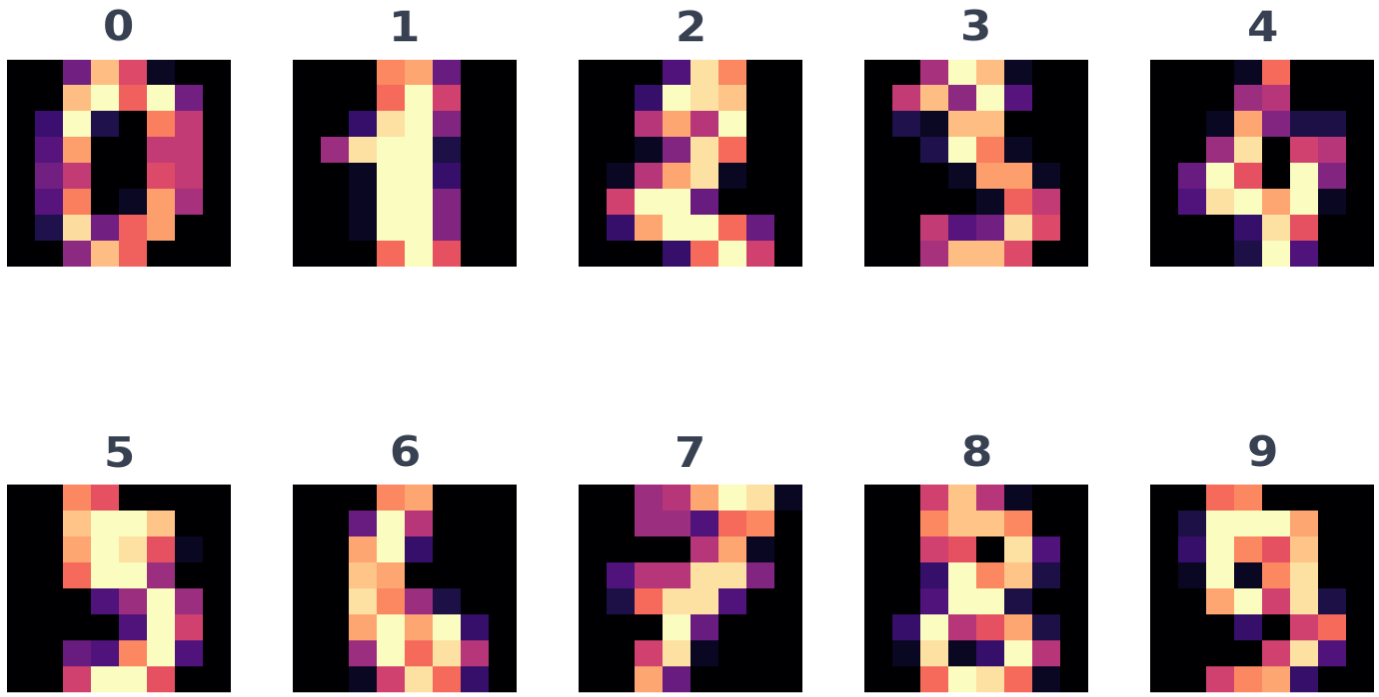
Wiele warstw liniowych nadal dałoby jedną liniową

```
g(x) = 2x + 1
h(z) = 3z + 2

h(g(x)) = 3(2x + 1) + 2 = 6x + 5
```

Sama „głębokość” nie wystarczy — potrzebujemy nieliniowości między warstwami.

Model musi się nauczyć swoich wag



→

etykiety:

0, 1, 2, …, 9

Dane uczą modelu, jakie wyniki powinny być poprawne.

Trening, walidacja i test — trzy różne zadania

Trening (70%)

Sieć widzi przykłady i aktualizuje parametry.

Walidacja (15%)

Sprawdzamy hiperparametry oraz kiedy przerwać trening.

Test (15%)

Oceniamy gotowy model na wcześniej niewidzianych danych.

Funkcja straty: jak bardzo myli się sieć?

Poprawna odpowiedź: 3

Model A: 70% na cyfrę 3



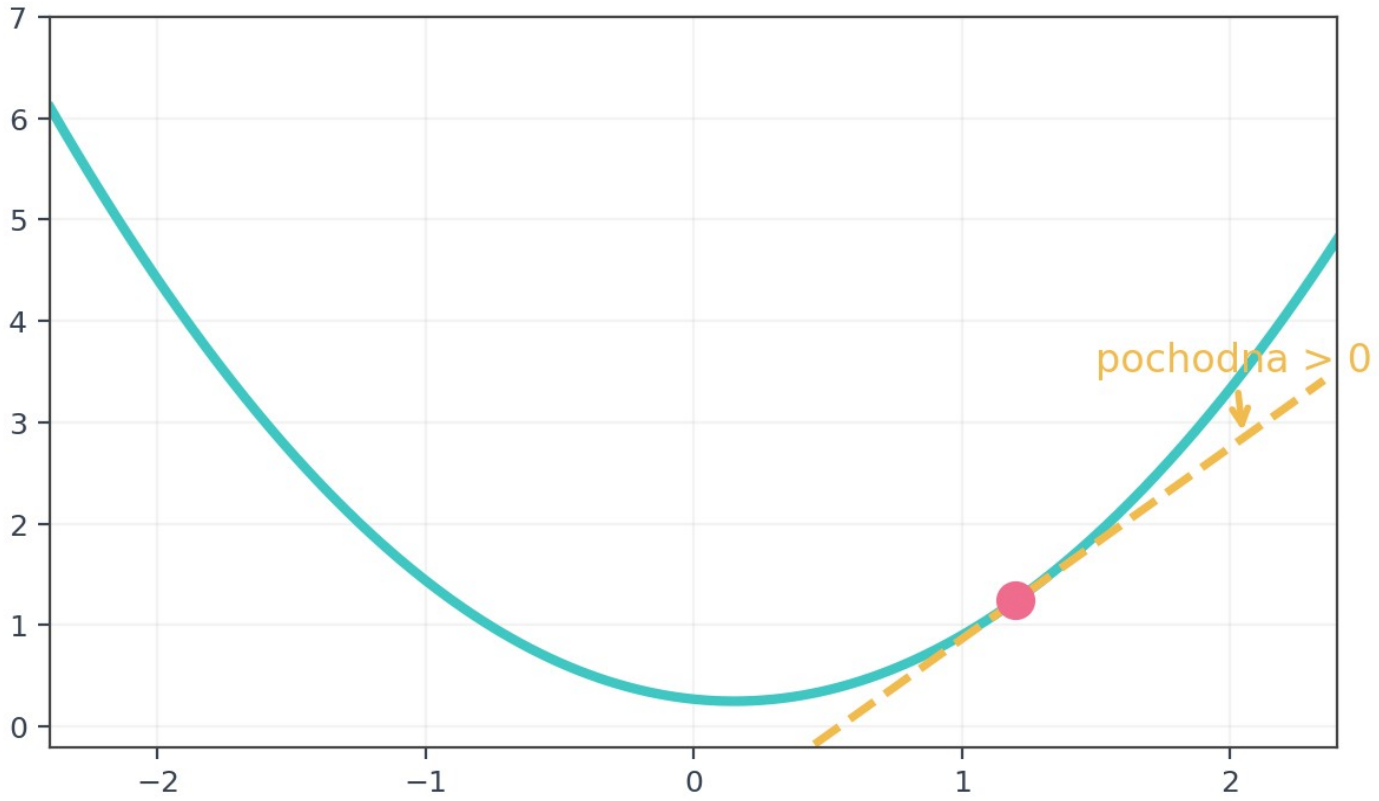
Model B: 25% na cyfrę 3

Model C: 1% na cyfrę 3

Cross-entropy

Gdy maleje prawdopodobieństwo poprawnej klasy, strata rośnie.

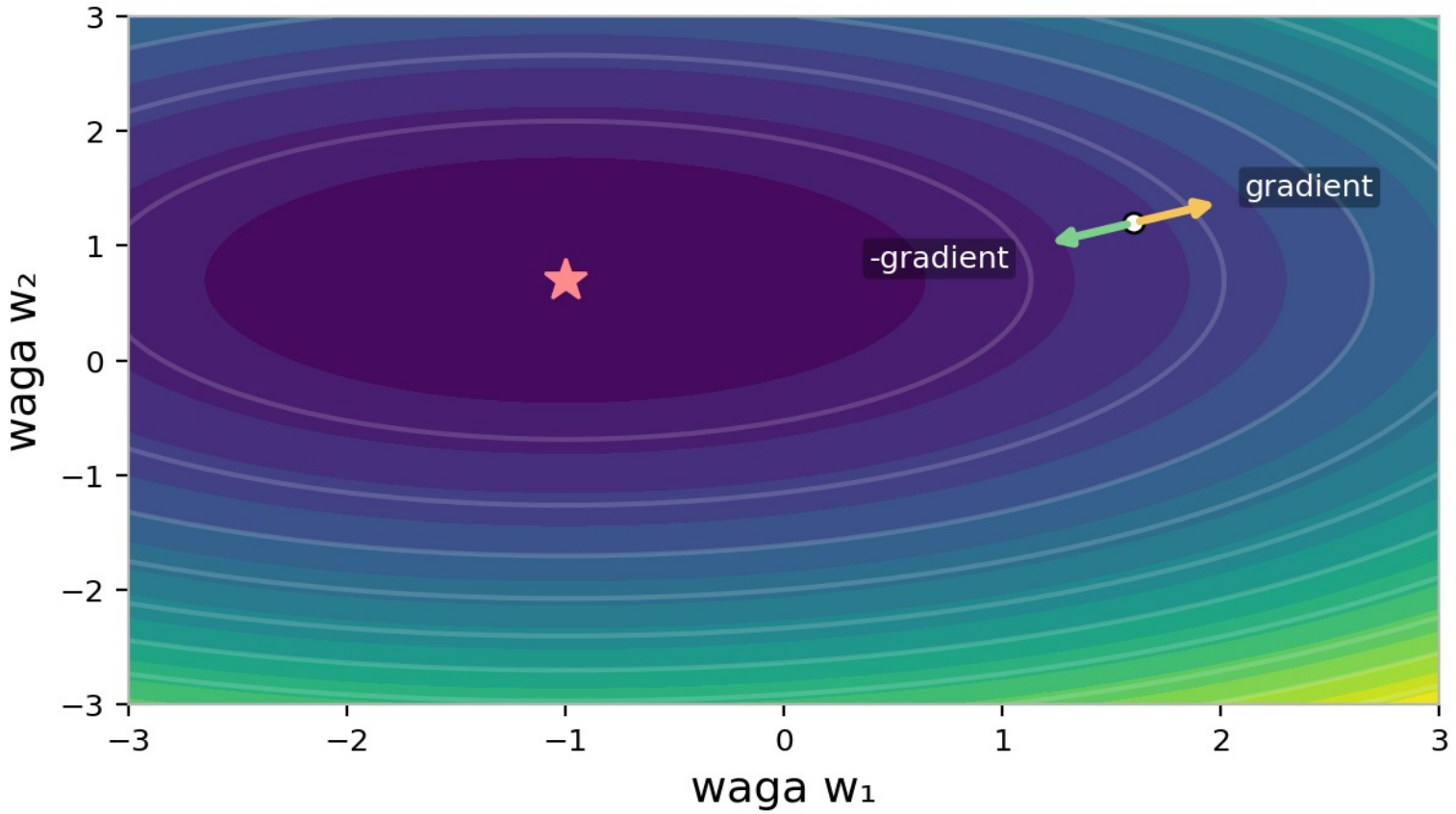
Pochodna: czy funkcja rośnie czy maleje?



Nachylenie

Pochodna mówi, jak zmieni się strata, gdy lekko zmienimy jeden parametr.

Gradient zbiera pochodne względem wszystkich wag



gradient

wektor wszystkich pochodnych

w kierunku − gradientu maleje strata

Gradient descent: krok w dół straty

```
NOWA WAGA = STARA WAGA − LR × GRADIENT
```

Gradient

W którą stronę?

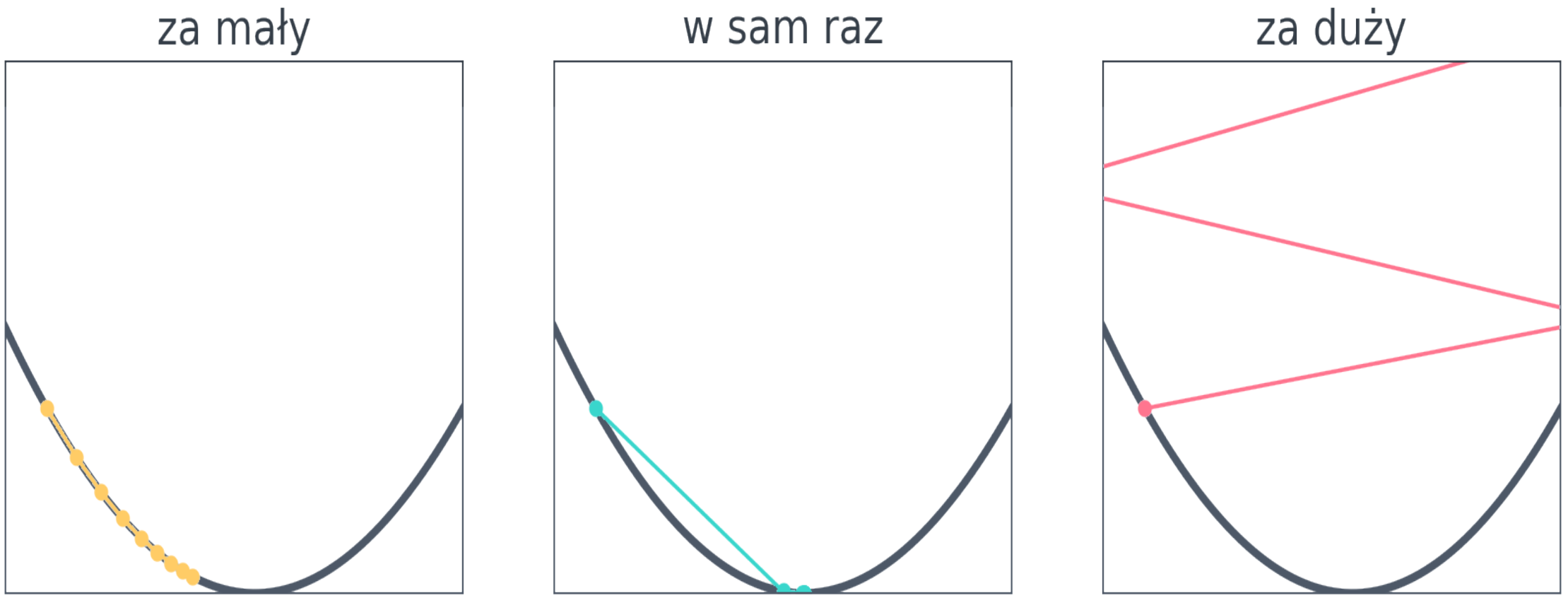
Learning rate

Jak duży krok?

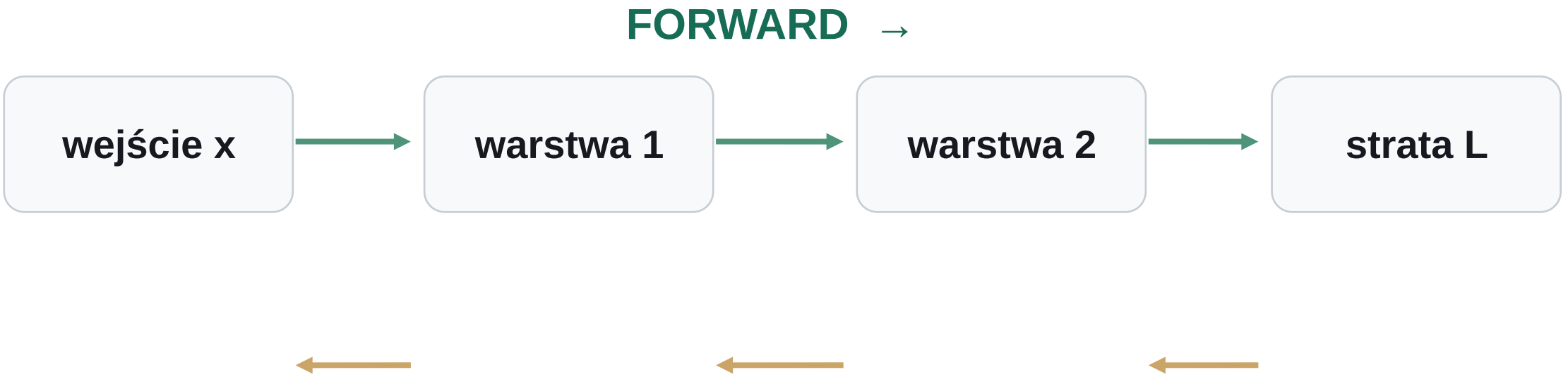
Powtarzamy

Aż model się nauczy.

Learning rate: za mały, w sam raz, za duży



Backpropagation: jak gradient wraca przez warstwy?



Backprop: intuicja na małym przykładzie

```
x = 2          w = 3          cel y = 10

predykcja = x × w = 6
strata = (predykcja − 10)² = 16
```

Czy wagę w warto zwiększyć, czy zmniejszyć?

Mini-batche: uczymy się na paczkach przykładów

Całe dane

Dużo obliczeń przed każdym krokiem.

„Full batch”

1 przykład

Częste, ale szumne aktualizacje.

„Stochastic”

64 przykłady

Praktyczny kompromis szybkości i jakości gradientu.

„Mini-batch”

Epoka, batch, iteracja — ważne nazwy

Dane treningowe

1 257 obrazków

Rozmiar mini-batcha

64 przykłady

Kroki na epokę

20 aktualizacji

20 epok ≈ 400 aktualizacji wag.

PyTorch: tensor = tablica liczb

```
import torch

x = torch.tensor([[1., 2.], [3., 4.]])
print(x.shape)              # torch.Size([2, 2])

# Tensor wspiera obliczenia i autograd.
```

Tworzenie MLP w kilku linijkach

```
from torch import nn

model = nn.Sequential(
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
)

logits = model(x)  # 10 wyników klas
```

Kontrola wymiarów: batch × liczba cech

Wejście	Warstwa ukryta	Wyjście
[32, 64]	[32, 64]	[32, 10]

32 obrazki × 10 wyników (cyfry 0–9)

Pętla treningowa: pięć najważniejszych operacji

```
optimizer.zero_grad()        # wyzeruj stare gradienty
logits = model(x_batch)     # forward
loss = criterion(logits, y_batch)
loss.backward()             # backprop
optimizer.step()            # aktualizacja wag
```

Autograd: PyTorch liczy pochodne automatycznie

```
w = torch.tensor(0., requires_grad=True)
loss = (w - 3)**2
loss.backward()
print(w.grad)    # -6
```

PyTorch śledzi operacje wykonywane na parametrach.

DataLoader sam przygotowuje mini-batche

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

for x_batch, y_batch in train_loader:
    # krok uczenia...
```

Trening to pętla powtarzana przez epoki

1. Trening

model.train()

Batches, forward, loss, backward, step.

2. Walidacja

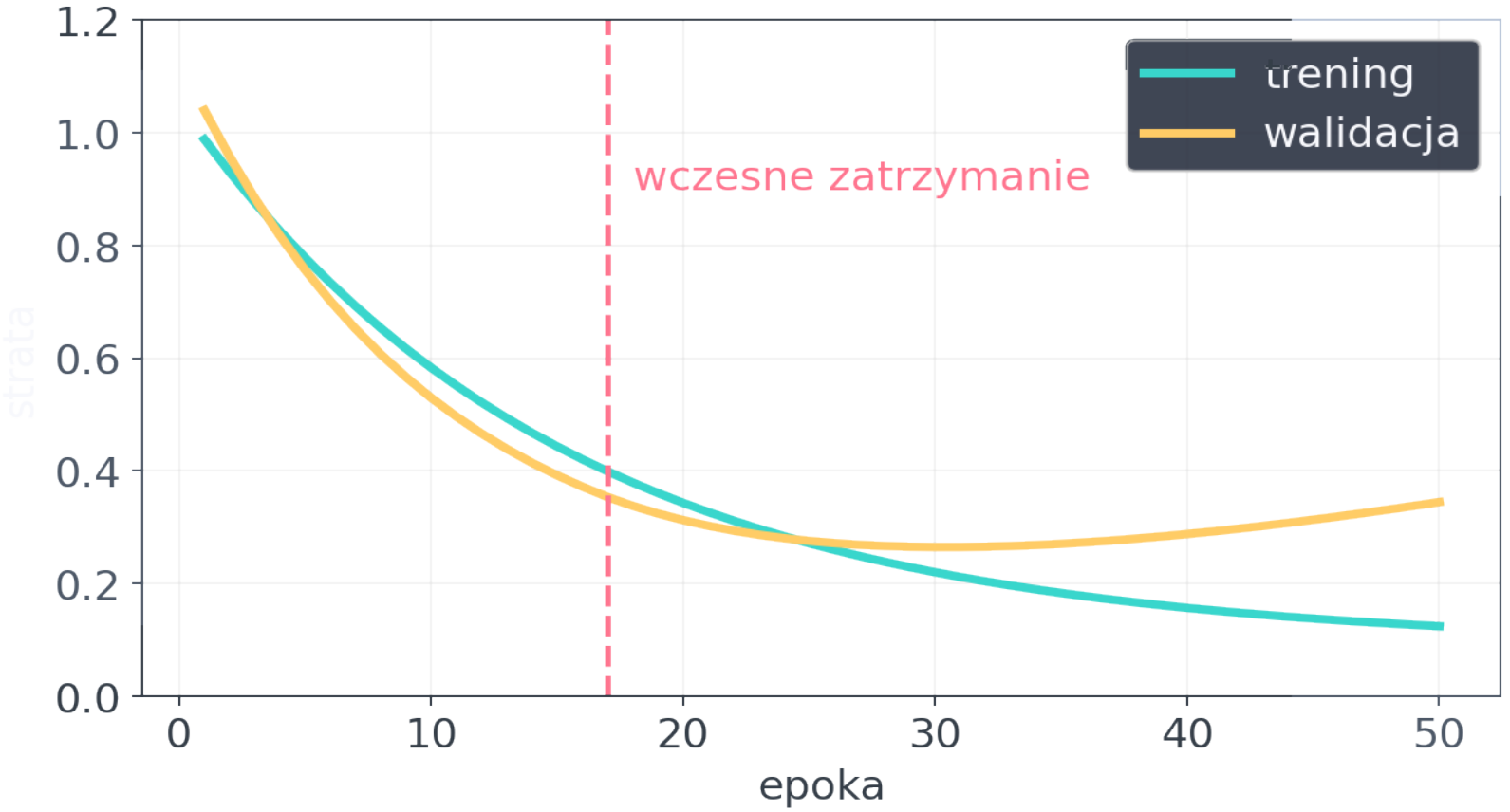
model.eval()
torch.no_grad()

Loss + accuracy.

3. Decyzja

Zapisz najlepszy model lub przerwij przy braku poprawy.

Uwaga na overfitting (przeuczenie)



Objaw

Świetny wynik na treningu, gorszy na nowych przykładach.

Trzy sposoby ograniczania przeuczenia

Dropout

Losowo wyłączamy aktywacje w trakcie treningu.

Weight decay

Ograniczamy wzrost wag; sprzyja prostszym rozwiązaniom.

Early stopping

Przerywamy, gdy walidacja przestaje się poprawiać.

Inicjalizacja i scheduler — krótkie narzędzia

Inicjalizacja wag

Wagi na starcie są zwykle losowe. Dobra skala (np. He dla ReLU) ułatwia uczenie; nie ustawiamy wszystkich wag na zero.

LR scheduler

Na początku większe kroki, później mniejsze. W notebooku co 10 epok mnożymy LR przez ustalony współczynnik.

SGD, RMSprop, Adam i AdamW — co wybrać?

SGD

Prosty, popularny punkt odniesienia. Z momentum pamięta kierunek.

RMSprop

Dostosowuje skalę kroku osobno dla parametrów.

Adam

Łączy „pamięć” kierunku i adaptacyjną skalę kroku.

AdamW

Adam + rozdzielone kurczenie wag (weight decay).

Gdy loss nie spada — co sprawdzić?

Loss = NaN lub rośnie	Zbyt duży learning rate → zmniejsz go 10×.
Loss stoi w miejscu	Sprawdź LR, etykiety, loss.backward() i optimizer.step().
Train dobry, val gorsza	Dropout, weight decay, early stopping.
Accuracy ≈ 10%	To poziom losowy dla 10 klas — sprawdź dane i kod.

Po wykładzie: trzy eksperymenty

Eksperyment A

Sprawdź lr = 0.0001, 0.003 i 0.1. Jak wyglądają krzywe strat?

Eksperyment B

Porównaj SGD, RMSprop, Adam i AdamW.

Eksperyment C

Wyłącz i włącz dropout, weight decay, early stopping.

4 rzeczy, które warto zapamiętać

01 / MLP

Wagi + bias + ReLU + warstwy.

02 / Loss

Liczba opisująca błąd modelu.

03 / Gradient

Kierunek poprawiania parametrów.

04 / PyTorch

forward → loss → backward → step.

Dziękuję za uwagę!